

Архитектура проекта «Научный клубок R&D»

Система представляет собой интеллектуальный поиск по графу знаний, построенному на базе научно-исследовательских статей, патентов и отчетов (PDF, DOCX, архивы) в области металлургии.

Ниже описано, из каких компонентов состоит система и как они взаимодействуют между собой.

1. Как работают потоки данных (Data Flow)

Сценарий А: Загрузка новых знаний (Индексация)

1. Пользователь загружает PDF-статью через левое меню Streamlit.
2. Файл сохраняется в папку `inbound/`.
3. Фоновый процесс `pipeline.py` видит новый файл, запускает PDF-парсер.
4. Текст нарезается на куски (`chunks`).
5. `NerPipeline` прогоняет текст через `spaCy`: извлекает узлы, строит связи внутри текста и вычисляет векторы (`embeddings`) для каждого узла.
6. Python формирует JSON файл (сохраняет в `done_json/`) и отправляет POST-запрос на Go-бэкенд (`/api/v1/entities`).
7. Go-бэкенд записывает узлы и ребра в Neo4j.

Сценарий Б: Поиск и генерация ответа

1. Пользователь вводит запрос: "никель" в Streamlit.
2. `Search` класс в Python векторизует слово "никель".
3. Вектор улетает на Go-бэкенд.
4. Go-бэкенд формирует запрос в Neo4j:
 - **Векторный поиск:** Находит узел, вектор которого максимально близок (по косинусному расстоянию) к запросу "никель". Дополнительно сортирует выдачу по количеству связей (`COUNT { (n)--() } DESC`), чтобы при наличии нескольких похожих узлов выбрать самый "центральный" хаб.

- **Графовый обход:** От найденного узла база делает обход в ширину на глубину от 1 до 4 шагов (MATCH path = (n)-[*1..4]-(m)).
5. Вся собранная паутина возвращается в Go.
 6. Go преобразует данные в DTO (Data Transfer Objects), очищает их от мусора и отдает в Streamlit.
 7. Streamlit рисует Pyvis-граф на экране и заполняет карточку свойств при клике.

2. Основные компоненты системы

2.1 Фронтенд и Оркестратор (Python + Streamlit)

Расположение: backend-python/app.py

Отвечает за весь пользовательский интерфейс (UI) и оркестрацию запросов:

- **Поисковая строка:** Принимает естественные запросы от пользователя (например, *"Очистка сточных вод сульфаты <250 мг/л"*).
- **Семантическая обработка:** Фронтенд локально векторизует текстовый запрос с помощью легковесной мультязычной NLP-модели (Sentence Transformers).
- **Отрисовка графа:** Принимает от Go-бэкенда плоский список узлов и связей, и с помощью библиотеки **Pyvis** генерирует интерактивный JavaScript-граф прямо в браузере (включая тултипы со свойствами, подкраску по типам и связям).
- **Аналитика:** Генерирует текстовые сводные Markdown-отчеты под графом на основе найденного подграфа.
- **Загрузка файлов:** Предоставляет UI для загрузки новых PDF/DOCX файлов, сохраняя их в папку `inbound/` для последующей асинхронной обработки.

2.2 Машинное обучение и Пайплайн загрузки данных (Python)

Расположение: backend-python/ml/ и backend-python/pipeline.py

Автономная система (ETL-пайплайн), которая "слушает" папку `inbound/`, читает файлы и превращает сырой текст в структурированный граф:

- **Парсеры (parser/):** Набор утилит для извлечения сырого текста из PDF (через `pdfminer`), DOCX, ODT и TXT.
- **NLP Движок (ner_extractor.py):** Ядро извлечения. Использует **SpaCy** (модели `ru_core_news_md` и `en_core_web_md`) в связке с кастомным компонентом `EntityRuler`.

- **Извлечение сущностей (NER):** На основе заранее подготовленных профильных металлургических словарей (`constants.py`) и регулярных выражений (RegEx) система находит:
 - Процессы (*выщелачивание, плавка*)
 - Материалы (*файнштейн, католит*)
 - Численные параметры (*температура 80 °C, концентрация < 200 мг/л*) с умным парсингом операторов (<, >, =).
 - Авторов, Заводы, Географию и Номера протоколов.
- **Построение связей:** Модель не просто находит слова, но и связывает их (например, Материал связан с Процессом через `uses_material` , если они находятся в одном предложении). Все найденные сущности программно связываются с общей сущностью Статьи (`Publication`) через связь `mentioned_in` .

2.3 Высоконагруженный Бэкенд (Go)

Расположение: `backend-go/`

Микросервис на базе фреймворка **Fiber**, написанный с использованием принципов Clean Architecture:

- Отвечает за быструю маршрутизацию запросов между Streamlit и базой данных Neo4j.
- **Эндпоинт** `/api/v1/entities` : Принимает огромные JSON-графы (сгенерированные в Python) и массово (batch) заливает их в Neo4j, создавая узлы и ребра.
- **Эндпоинт** `/api/v1/search` : Принимает поисковый вектор от Streamlit и транслирует его в сложный графовый запрос **Cypher**. Бэкенд на лету фильтрует технические поля (например, убирает массивы векторов `embedding` из выдачи) для экономии трафика перед отправкой на фронтенд.

2.4 Графовая база данных (Neo4j)

Хранит всю онтологию. Каждый узел в базе содержит свои свойства и, самое главное, `embedding` (числовой вектор смысла).